

31. What happens when features are highly correlated in Lasso?

When you have highly correlated features (multicollinearity), Lasso regression (L_1 regularization) exhibits a unique and unstable behavior:

- **Arbitrary Elimination:** If three or four features are highly correlated and all carry a similar predictive signal, Lasso's geometric constraint forces it to randomly pick **one** feature from that group to do all the heavy lifting. It then drives the weights of all the other remaining correlated features to **exactly zero**.
- **High Sensitivity:** Because this selection is highly sensitive to minor statistical fluctuations, tiny shifts or random noise in your training dataset can cause Lasso to abruptly drop its previously chosen feature and pick a different correlated one instead.
- **Loss of Group Dynamics:** Unlike Ridge regression—which keeps all correlated features and shrinks their weights proportionally together—Lasso breaks up the group, making model interpretation highly volatile when dealing with highly collinear data.

32. What is the advantage of combining L1 and L2 penalties in Elastic net?

Elastic Net combines the penalties of both Lasso (L_1) and Ridge (L_2) regression into a single cost function.

The Core Advantages

1. **The Grouping Effect:** By incorporating the L_2 (Ridge) penalty, Elastic Net fixes Lasso's weakness with correlated variables. Instead of arbitrarily picking one feature and discarding the rest, it encourages highly correlated features to be shrunk and selected **together** as a cohesive group.
2. **Preserving Sparsity:** Thanks to its L_1 component, it retains Lasso's ability to completely zero out entirely irrelevant features, keeping your final model simple and clean.
3. **Handling High-Dimensional Data:** In scenarios where you have far more features (n) than training samples (m), Lasso is mathematically capped at selecting at most m features. Elastic Net breaks through this ceiling, allowing you to safely evaluate all available features without the model breaking down structurally.

33. What is K-Means clustering? How are initial centroids chosen in K-Means? Why do we iterate in K-Means? Why is K-Means sensitive to initial centroids? Why do we need feature

scaling in K-Means? What type of cluster shapes does K-Means work well on?

Qualitative Definition

K-Means is an unsupervised clustering algorithm designed to split a dataset into a pre-defined number (K) of distinct, non-overlapping groups based on spatial proximity.

Centroid Initialization

- **Random Initialization:** Standard K-Means picks K random data points from the dataset to act as the initial starting centers (centroids).
- **K-Means++:** To prevent bad placements, K-Means++ picks the first center at random, and then spaces out the remaining centers by prioritizing points that are furthest away from the existing ones.

Why We Iterate

Finding the absolute perfect cluster configuration across millions of points simultaneously is mathematically impossible to compute directly (NP -hard). K-Means uses an iterative loop to gradually crawl toward a clean solution:

1. **Assignment Step:** Every data point looks at the current centroids and attaches itself to the closest one.
2. **Update Step:** Each centroid recalculates its position by moving to the exact geometric average (mean) of all points currently assigned to it. This loop repeats until the centroids completely stop moving, marking a stable local equilibrium.

Sensitivity to Initialization

Because the algorithm only moves centroids based on local averages, its final result is entirely dependent on where you start. If you place two initial centroids inside the same natural data group, the algorithm will get trapped in a poor local minimum, splitting a single coherent cluster in half while mixing other distinct groups together elsewhere.

Why Feature Scaling is Mandatory

K-Means calculates straight-line distances to group data. If one feature spans a massive range (like house prices from \$100,000 to \$1,000,000) and another spans a tiny range (like bedrooms from 1 to 5), the distance formula will be completely overwhelmed by the large-scale feature. Scaling balances all features onto an equal playing field so they contribute equally to the geometric grouping.

Ideal Cluster Shapes

K-Means works well only on clusters that are **spherical (globular), compact, and roughly equal in size and density**. It fails completely on elongated shapes, complex curves, or concentric rings because its distance metric naturally slices up the space using straight lines.

34. What is the difference between DBSCAN and K-Means? What is ϵ (epsilon) in DBSCAN? Why is DBSCAN good for detecting outliers?

Core Differences

Property	K-Means	DBSCAN
Core Concept	Center-based: Groups data by minimizing the distance of points to a central anchor.	Density-based: Groups continuous regions of packed data together, ignoring empty space.
Cluster Shape	Can only find spherical, convex shapes well.	Easily finds highly complex, winding, or arbitrary geometric shapes.
Defining K	You must explicitly tell it how many clusters to find beforehand.	It automatically discovers the number of clusters based on the natural layout.
Outliers	Highly sensitive; outliers will distort the centroids.	Completely immune; it isolates anomalies and flags them as noise.

The Epsilon (ϵ) Parameter

In DBSCAN, ϵ (**epsilon**) defines the radius of the search circle around any given point. It sets the physical boundary within which the algorithm counts neighboring points to measure local data density.

Why DBSCAN is Ideal for Outlier Detection

DBSCAN defines a true cluster as a region where data points are packed tightly together. To be part of a cluster, a point must have a minimum number of neighbors within its ϵ circle. If an anomalous data point sits in a lonely, sparse region and doesn't meet this threshold, DBSCAN leaves it unassigned, explicitly labeling it as **noise**. This makes it an excellent tool for automatically filtering noise out of experimental data.

35. What are the types of encoding that you know? When should we use one-hot encoding instead of label encoding? What problem can arise from label encoding?

Common Encoding Types

- **Label Encoding:** Replaces each text category with a sequential number (e.g., Red $\rightarrow$ 0, Green $\rightarrow$ 1, Blue $\rightarrow$ 2).
- **One-Hot Encoding (OHE):** Creates a brand new binary column for every unique category, filling it with a 1 if the row matches that category and 0 otherwise.

When to Use One-Hot Encoding

You should use One-Hot Encoding for **nominal variables**—categories that have **no natural order, rank, or hierarchy** (e.g., countries, animal species, or colors). Label encoding should be reserved strictly for **ordinal variables** that possess an inherent sequence (e.g., Low $\rightarrow$ 0, Medium $\rightarrow$ 1, High $\rightarrow$ 2).

The Core Problem with Label Encoding Nominal Data

If you apply label encoding to nominal data like colors, you accidentally introduce an artificial mathematical order that does not exist in the real world (e.g., Red (0) < Green (1) < Blue (2)). Distance-based or linear models will misinterpret these numbers as continuous measurements, incorrectly assuming that Blue (2) is twice as large or twice as important as Green (1), leading to distorted fits and poor predictions.

36. What is feature scaling? Which algorithms are sensitive to feature scaling?

Qualitative Definition

Feature scaling is a data preprocessing step that standardizes or normalizes the numerical ranges of your variables, bringing them all onto an identical scale without altering the underlying relationships or distributions in your data.

Sensitive Algorithms

- **Distance-Dependent Models:** Algorithms like *k*-Nearest Neighbors (KNN) and K-Means Clustering rely completely on geometric distance calculations. Unscaled variables with massive numeric ranges will completely dominate the equations, rendering smaller-scale features invisible.
- **Gradient Descent Optimizers:** In models like Logistic Regression and Neural Networks, vastly different scales warp the error landscape into an elongated, asymmetric valley. This causes optimization updates to oscillate wildly back and forth, severely delaying or preventing convergence.
- **Support Vector Machines (SVM):** SVMs maximize the physical margin separating classes. If features are unscaled, the support vector coordinates will warp the orientation of the

separating hyperplane along the axes of the largest numerical ranges.

Immune Algorithms: Tree-based models (like Decision Trees and Random Forests) are completely invariant to feature scaling. They isolate and split one feature at a time, making their mathematical partitions independent of the scales of other variables.

37. What is normalization (Min-Max scaling)? What is standardization (Z-score scaling)? What happens if features are not scaled?

Normalization (Min-Max Scaling)

Normalization compresses all your data points so they fall within a strict, bounded range, typically between 0 and 1. It works by finding the absolute minimum and maximum values of a feature and scaling the data proportionally.

- **Best use:** Excellent when you need bounded limits (like image pixels scaled between 0 and 1) or when you know your data does not follow a bell curve.

Standardization (Z-Score Scaling)

Standardization centers your data around a mean of 0 and rescales it so that its standard deviation equals 1. It does not clamp your data into a rigid box.

- **Best use:** Highly preferred when your machine learning algorithms assume normally distributed data, or when your data contains extreme outliers, because it won't crush outliers into an uninformative, tiny fraction like Min-Max scaling does.

What Happens If Features Are Left Unscaled

Leaving features unscaled warps your model's perspective. Optimization loops will run incredibly slowly or fail to find the true error valley, distance-based classifiers will make poor decisions by ignoring lower-magnitude variables, and regularization methods (like Ridge or Lasso) will unfairly penalize features simply because of the units they are measured in.

38. What is multicollinearity? Why is multicollinearity a problem in regression? How can multicollinearity be detected? Why does multicollinearity not always affect prediction accuracy? How can correlated features be detected?

Qualitative Definition

Multicollinearity is a situation where two or more independent features in your dataset are highly correlated with each other. Essentially, they are carrying the exact same information, making them redundant.

Why it is a Severe Problem for Regression

1. **Unstable Model Weights:** It completely destabilizes the mathematical calculation of your regression coefficients. Because the features mirror each other, the model cannot figure out which feature is actually driving the target variable, causing the weights to swing erratically based on tiny shifts in the data.
2. **Loss of Interpretability:** It destroys your ability to explain the model. You cannot isolate the independent impact of a single feature on your target because its twin variable always moves along with it.

Detection Methods

- **Correlation Heatmaps:** Visually inspecting a correlation matrix to flag pairs with coefficients approaching ± 1 .
- **Variance Inflation Factor (VIF):** A score that measures how much a feature's weight variance is inflated by the presence of collinearity. A VIF score above 5 or 10 indicates a problem.

Impact on Prediction Accuracy

Interestingly, multicollinearity **does not damage overall prediction accuracy**. If the tightly coupled relationships between your features remain exactly the same in both your training data and your new testing data, the model can still generate highly accurate predictions. The issue is structural and explanatory: it ruins the stability and meaning of your weights, but the collective predictive signal remains intact.

39. What is eigenvalue and eigenvector in PCA? What are the steps involved in PCA? How does PCA reduce multicollinearity? Can PCA handle nonlinear data? When would you prefer feature selection over PCA?

Eigenvalues and Eigenvectors in PCA

Principal Component Analysis (PCA) simplifies high-dimensional datasets by projecting them onto new, synthetic axes.

- **Eigenvector:** The spatial direction of the new axis (the Principal Component) along which the data is spread out.

- **Eigenvalue:** A number that quantifies exactly how much data variance (information) is retained along that specific eigenvector axis.

Step-by-Step PCA Mechanics

- **Step 1: Standardization:** Scale all features to have a mean of 0 and a standard deviation of 1 so large-scale features don't hijack the variance calculations.
- **Step 2: Covariance Matrix:** Construct a matrix to map out how all feature pairs vary together.
- **Step 3: Eigendecomposition:** Extract the eigenvectors and eigenvalues from that matrix.
- **Step 4: Sorting:** Sort the eigenvectors in descending order based on their eigenvalues (highest variance down to lowest).
- **Step 5: Projection:** Drop the lowest-ranked components and multiply your original data by the remaining top eigenvectors to project the data into a simpler, lower-dimensional space.

Reducing Multicollinearity

PCA targets the root cause of multicollinearity by design. Because eigenvectors are mathematically engineered to be perfectly orthogonal (perpendicular) to one another, **the resulting principal components are completely uncorrelated**. Replacing your original collinear variables with these components eliminates multicollinearity entirely.

Handling Nonlinear Data

Standard PCA is a strictly **linear transformation** and cannot grasp curved, winding relationships. To handle non-linear data structures, you must use **Kernel PCA**, which leverages the kernel trick to map data into a higher dimensional space where linear separations become possible.

Choosing Feature Selection Over PCA

You should pick traditional feature selection (like Lasso) over PCA when:

- **Interpretability is Essential:** PCA blends your features into abstract linear combinations, making it impossible to explain what a component means in the real world. Feature selection leaves your original variables intact.
- **Data Collection Constraints:** PCA still requires you to collect all original features just to calculate the components, whereas feature selection allows you to stop measuring the dropped features entirely.

40. What are the main components of a neural network? What is the role of weights and biases? What is a layer in a neural

network? What are input, hidden, and output layers? What is backpropagation? What is loss function in neural networks for regression and classification problems?

Main Components

A Neural Network consists of basic processing nodes called **neurons** organized into sequential, parallel groups called **layers**.

Role of Weights and Biases

- **Weights:** Multiplicative values that act as volume dials. They control how much signal or importance an incoming feature has as it passes into a neuron.
- **Biases:** Additive values that act as an offset dial. They shift the activation function up or down, allowing the neuron to trigger even if the incoming inputs are zero.

The Layer Hierarchy

- **Input Layer:** The entry point that takes raw numbers directly from your dataset and passes them into the network.
- **Hidden Layers:** The internal processing layers. They take the basic inputs and extract abstract, high-level features and non-linear patterns.
- **Output Layer:** The final layer that translates the network's abstract internal signals into a concrete prediction (like a continuous number or a class probability).

Backpropagation

Backpropagation is the learning engine of a neural network. It looks at the final prediction error at the output layer and uses the calculus **chain rule** to pass that error signal backward through the hidden layers. It calculates exactly how much blame each weight and bias carries for the mistake, allowing an optimizer to tweak them to improve the next prediction.

Loss Functions

- **For Regression:** Mean Squared Error (MSE), which measures the squared physical distance to the true values.
- **For Classification:** Cross-Entropy Loss, which penalizes the model based on how far its predicted probabilities deviate from the true class labels.

41. What is an activation function? What is epoch, batch size, and iteration in NN? What is chain rule in backpropagation? Why do we compute derivatives in training?

Purpose of Activation Functions

An **activation function** is a mathematical formula applied to the combined sum inside a neuron. Without activation functions, a neural network is just a series of linear calculations, which means it can only fit a straight line, no matter how many hidden layers you add.

Activation functions inject **non-linearity**, giving the network the flexibility to bend and warp its decision paths to learn highly complex, real-world patterns.

Training Parameters

- **Epoch:** One complete cycle where the model looks at every single sample in your training dataset exactly once.
- **Batch Size:** The number of samples the model reviews at once before pausing to calculate gradients and update its weights.
- **Iteration:** A single weight update pass. For example, if you have 1,000 samples and your batch size is 100, it takes 10 iterations to finish 1 full epoch.

The Chain Rule in Backpropagation

Because a neural network is built like a set of nested Russian nesting dolls (inputs pass through layer after layer), changing a weight in an early layer ripples through the entire network. The **chain rule** is a calculus tool that lets us break down the final output error into a chain of individual multiplications, tracking exactly how an early parameter change impacts the final loss.

Why We Compute Derivatives

We compute derivatives because they act as a **compass**. A derivative calculates the exact gradient (slope) of the loss function at your current parameter state. By knowing whether the slope is positive or negative, the model knows exactly which direction to adjust its weights (up or down) to walk downhill toward the lowest possible error.